1. 프로젝트 핵심: 트랜잭션 & 동시성 (1~10)

1. **트랜잭션(Transaction)이란 무엇인가요?**
   - **답변:** 데이터베이스의 상태를 변화시키는 하나의 논리적 작업 단위입니다. ACID(원자성, 일관성, 고립성, 지속성) 성질을 가집니다.

2. **ACID 중 Isolation(고립성)과 격리 수준에 대해 설명해주세요.**
   - **답변:** 동시에 실행되는 트랜잭션들이 서로 얼마나 고립되어 있는지를 뜻합니다. Read Uncommitted, Read Committed(대부분 기본), Repeatable Read, Serializable 단계가 있습니다.

3. **이커머스 프로젝트에서 재고 감소 시 동시성 문제를 어떻게 해결했나요?**
   - **답변:** DB 의 **Pessimistic Lock(비관적 락)**을 사용해 정합성을 보장하거나, **Optimistic Lock(낙관적 락)**으로 충돌 발생 시 재시도 로직을 구현했습니다.

4. **Redis 를 동시성 제어에 활용한다면 어떻게 할 수 있나요?**
   - **답변:** Redisson 라이브러리를 사용해 **분산 락(Distributed Lock)**을 구현하여 여러 서버 인스턴스 환경에서도 자원 점유를 제어할 수 있습니다.

5. **낙관적 락(Optimistic Lock)과 비관적 락(Pessimistic Lock)의 차이는?**
   - **답변:** 낙관적 락은 충돌이 적을 것을 가정해 버전 관리(Version)를 사용하고, 비관적 락은 실제 DB row 에 락을 걸어 충돌을 원천 차단합니다.

6. **데드락(Deadlock)이란 무엇이며 어떻게 방지하나요?**
   - **답변:** 두 개 이상의 트랜잭션이 서로의 자원 락이 풀리기를 무한히 기다리는 상태입니다. 트랜잭션 진행 방향을 일정하게 설계하거나 대기 시간을 설정해 방지합니다.

7. **@Transactional 어노테이션의 동작 원리는?**
    - **답변:** Spring AOP 를 이용해 프록시 객체를 생성하고, 메서드 실행 전후로 트랜잭션을 시작하고 커밋/롤백합니다.

8. **트랜잭션 전파 속성(Propagation) 중 REQUIRED 와 REQUIRES_NEW 의 차이는?**
    - **답변:** REQUIRED 는 기존 트랜잭션에 합류하고, REQUIRES_NEW 는 항상 새 트랜잭션을 만들어 독립적으로 수행합니다.

9. **Checked Exception 발생 시 트랜잭션 롤백이 되나요?**
    - **답변:** 기본적으로 Spring 은 Unchecked Exception(RuntimeException)만 롤백합니다. Checked Exception 도 하려면 rollbackFor 옵션이 필요합니다.

10. **DB 커넥션 풀(DBCP)이란 무엇이며 왜 사용하나요?**
    - **답변:** 미리 DB 연결을 생성해두고 재사용하는 바구니입니다. 연결 생성 비용을 줄여 성능을 최적화합니다. (예: HikariCP)
    -

---

2. JPA & DB 성능 최적화 (11~20)

11. **JPA 를 사용하는 이유가 무엇인가요?**
    - **답변:** 객체 지향적 프로그래밍을 가능하게 하며, 생산성을 높이고 유지보수를 쉽게 합니다. 특정 DB 에 종속되지 않습니다.

12. **영속성 컨텍스트(Persistence Context)의 이점은?**
    - **답변:** 1 차 캐시, 동일성 보장, 쓰기 지연, 변경 감지(Dirty Checking), 지연 로딩을 지원합니다.

13. **N+1 문제란 무엇이며 어떻게 해결하나요?**

- **답변:** 연관 관계가 설정된 엔티티 조회 시, 조회된 데이터 개수만큼 추가 쿼리가 발생하는 현상입니다. **Fetch Join** 이나 **EntityGraph** 로 해결합니다.

14. **지연 로딩(Lazy)과 즉시 로딩(Eager)의 차이는?**
    - **답변:** 지연 로딩은 실제 사용 시점에 조회하고, 즉시 로딩은 연관 데이터를 한 번에 가져옵니다. 실무에서는 N+1 방지를 위해 지연 로딩을 권장합니다.

15. **Fetch Join 과 일반 Join 의 차이는?**
    - **답변:** 일반 Join 은 SELECT 절에 연관 엔티티를 포함하지 않지만, Fetch Join 은 연관 엔티티까지 한 번에 영속화합니다.

16. **영속성 컨텍스트의 '변경 감지(Dirty Checking)'는 어떻게 동작하나요?**
    - **답변:** 트랜잭션 커밋 시점에 스냅샷과 현재 엔티티를 비교하여 변경된 부분이 있으면 UPDATE 쿼리를 자동 생성합니다.

17. **DB 인덱스(Index)란 무엇이며 어떤 원리로 작동하나요?**
    - **답변:** 데이터 조회 성능을 높이기 위한 색인입니다. 주로 B-Tree 구조를 사용해 검색 범위를 좁힙니다.

18. **Composite Index(복합 인덱스) 설정 시 주의할 점은?**
    - **답변:** 인덱스 컬럼의 순서가 중요합니다. 카디널리티(중복도가 낮은 것)가 높은 컬럼을 앞에 두는 것이 유리합니다.

19. **JPA 의 save()와 saveAndFlush()의 차이는?**
    - **답변:** save 는 쓰기 지연 저장소에 보관했다가 커밋 시 반영하고, flush 는 즉시 DB 에 쿼리를 보냅니다(트랜잭션 커밋은 별개).

20. **Querydsl 을 사용하는 이유는?**
    - **답변:** 컴파일 시점에 문법 오류를 잡을 수 있고, 동적 쿼리 작성이 매우 편리하기 때문입니다.

---

3. 웹 & 인프라 (Redis, JWT, REST) (21~35)

21. **REST API 란 무엇이며 특징은?**
    - **답변:** 자원(URI), 행위(Method), 표현(Message)으로 구성된 아키텍처입니다. Stateless(무상태성)가 큰 특징입니다.

22. **GET 과 POST 의 차이는?**
    - **답변:** GET 은 리소스 조회(멱등성), POST 는 리소스 생성(비멱등성)에 사용합니다. GET 은 바디가 없고 URL 에 파라미터가 노출됩니다.

23. **Session 기반 인증 vs JWT 기반 인증의 차이는?**
    - **답변:** 세션은 서버 메모리에 상태를 저장(Stateful)하고, JWT 는 클라이언트가 토큰을 들고 있는 방식(Stateless)입니다. 확장성은 JWT 가 좋습니다.

24. **JWT 의 Access Token 과 Refresh Token 을 나누는 이유는?**
    - **답변:** 탈취 위험 때문입니다. 수명이 짧은 Access Token 으로 보안을 강화하고, 만료 시 Refresh Token 으로 재발급합니다.

25. **Redis 를 캐시로 사용할 때의 이점은?**
    - **답변:** 메모리 기반 저장소라 읽기/쓰기 속도가 매우 빠르며, DB 부하를 줄일 수 있습니다.

26. **캐시 전략 중 Look Aside(Cache Aside) 패턴이란?**

- **답변:** 데이터를 찾을 때 캐시를 먼저 보고, 없으면 DB 에서 가져온 뒤 캐시에 저장하는 가장 일반적인 전략입니다.

27. **HTTP 상태 코드 200, 201, 400, 401, 403, 404, 500 의 의미는?**
    - **답변:** 성공, 생성됨, 잘못된 요청, 인증 안됨, 권한 없음, 리소스 없음, 서버 에러.

28. **CORS(Cross-Origin Resource Sharing)란?**
    - **답변:** 출처(도메인)가 다른 서버 간의 리소스 공유를 허용할지 결정하는 브라우저 보안 정책입니다.

29. **웹 소켓(Web Socket)과 HTTP 의 차이는?**
    - **답변:** HTTP 는 단방향 요청-응답 구조이고, 웹 소켓은 한 번 연결하면 실시간 양방향 통신이 가능한 프로토콜입니다.

30. **로드 밸런서(Load Balancer)란 무엇인가요?**
    - **답변:** 여러 대의 서버에 트래픽을 균등하게 분산시켜주는 장치나 소프트웨어입니다.

31. **HTTPS 와 SSL/TLS 에 대해 설명해주세요.**
    - **답변:** HTTP 에 암호화 레이어(SSL/TLS)를 추가해 데이터를 안전하게 주고받는 프로토콜입니다.

32. **API 멱등성(Idempotency)이란 무엇인가요?**
    - **답변:** 같은 요청을 여러 번 보내도 결과(서버 상태)가 같은 성질입니다. GET, PUT, DELETE 는 멱등해야 합니다.

33. **CI/CD 란 무엇인가요?**

- **답변:** 지속적 통합(CI)과 지속적 배포(CD)를 통해 자동화된 빌드, 테스트, 배포를 수행하는 프로세스입니다.

34. **도커(Docker)를 사용하는 이유는?**
    - **답변:** 어느 환경에서나 동일한 설정으로 애플리케이션을 실행(컨테이너화)할 수 있기 때문입니다.

35. **무중단 배포란 무엇인가요?**
    - **답변:** 서비스 중단 없이 새 버전을 배포하는 기술입니다. (Blue-Green, Rolling, Canary 배포 등)

---

4. Java & Spring 프레임워크 (36~50)

36. **OOP(객체지향 프로그래밍)의 4 가지 특징은?**
    - **답변:** 캡슐화, 상속, 다형성, 추상화입니다.

37. **다형성(Polymorphism)이 프로젝트에서 어떻게 쓰였나요?**
    - **답변:** 인터페이스(예: PaymentService)를 두고 다양한 구현체(KakaoPay, TossPay)를 갈아 끼울 수 있도록 설계했습니다.

38. **JVM 의 메모리 구조(Heap, Stack)에 대해 설명해주세요.**
    - **답변:** Heap 은 동적으로 생성된 객체가 저장되고 GC 의 관리 대상입니다. Stack 은 지역 변수와 메서드 호출 정보가 저장됩니다.

39. **GC(Garbage Collection)란 무엇인가요?**
    - **답변:** Heap 영역에서 참조되지 않는 객체들을 찾아 메모리에서 해제하는 자동 프로세스입니다.

40. **==와 equals()의 차이는?**
    - **답변:** ==는 주소값을 비교하고, equals()는 객체 내부의 값을 비교합니다.

41. **인터페이스와 추상 클래스의 차이는?**
    - **답변:** 인터페이스는 '할 수 있는 기능'의 명세(다중 상속 가능)이고, 추상 클래스는 '부모의 특성을 확장'하는 것(단일 상속)입니다.

42. **DI(Dependency Injection)란 무엇이며 왜 사용하나요?**
    - **답변:** 의존 객체를 외부에서 주입받는 방식입니다. 결합도를 낮추고 유연성과 테스트 용이성을 높입니다.

43. **IoC(Inversion of Control) 제어의 역전이란?**
    - **답변:** 객체의 생성 및 생명주기 관리 권한을 개발자가 아닌 스프링 컨테이너가 가져가는 것입니다.

44. **스프링 빈(Bean)의 생명주기에 대해 설명해주세요.**
    - **답변:** 객체 생성 -> 의존 설정 -> 초기화 콜백 -> 사용 -> 소멸 콜백 순입니다.

45. **스프링 빈의 기본 스코프는 무엇인가요?**
    - **답변:** 싱글톤(Singleton)입니다. 애플리케이션당 하나의 객체만 생성되어 공유됩니다.

46. **DispatcherServlet 의 역할은?**
    - **답변:** 중앙 서블릿으로서 모든 HTTP 요청을 받아 적절한 컨트롤러로 배분해줍니다.

47. **AOP(Aspect Oriented Programming)란?**

- **답변:** 공통 관심사(로깅, 트랜잭션 등)를 핵심 비즈니스 로직에서 분리하여 재사용하는 프로그래밍 기법입니다.

48. **Wrapper Class 와 Primitive Type 의 차이는?**
    - **답변:** Primitive 는 순수 값만 저장하고, Wrapper 는 객체화하여 null 을 허용하거나 메서드를 사용할 수 있습니다.

49. **람다(Lambda)와 스트림(Stream) API 의 장점은?**
    - **답변:** 가독성을 높이고 데이터를 선언적으로 처리할 수 있으며, 병렬 처리가 용이합니다.

50. **JUnit 을 사용한 단위 테스트(Unit Test)의 중요성은?**
    - **답변:** 작은 단위의 기능이 정상 작동함을 보장하여 리팩토링 시 안정성을 제공하고 코드 품질을 높입니다.